



◆ CS EDUCATION ◆



# REST API



24.03.22  
박세연 감자



# 이제 곧 시작될 프로젝트, 백엔드와 프론트엔드는 어떻게 통신하지?

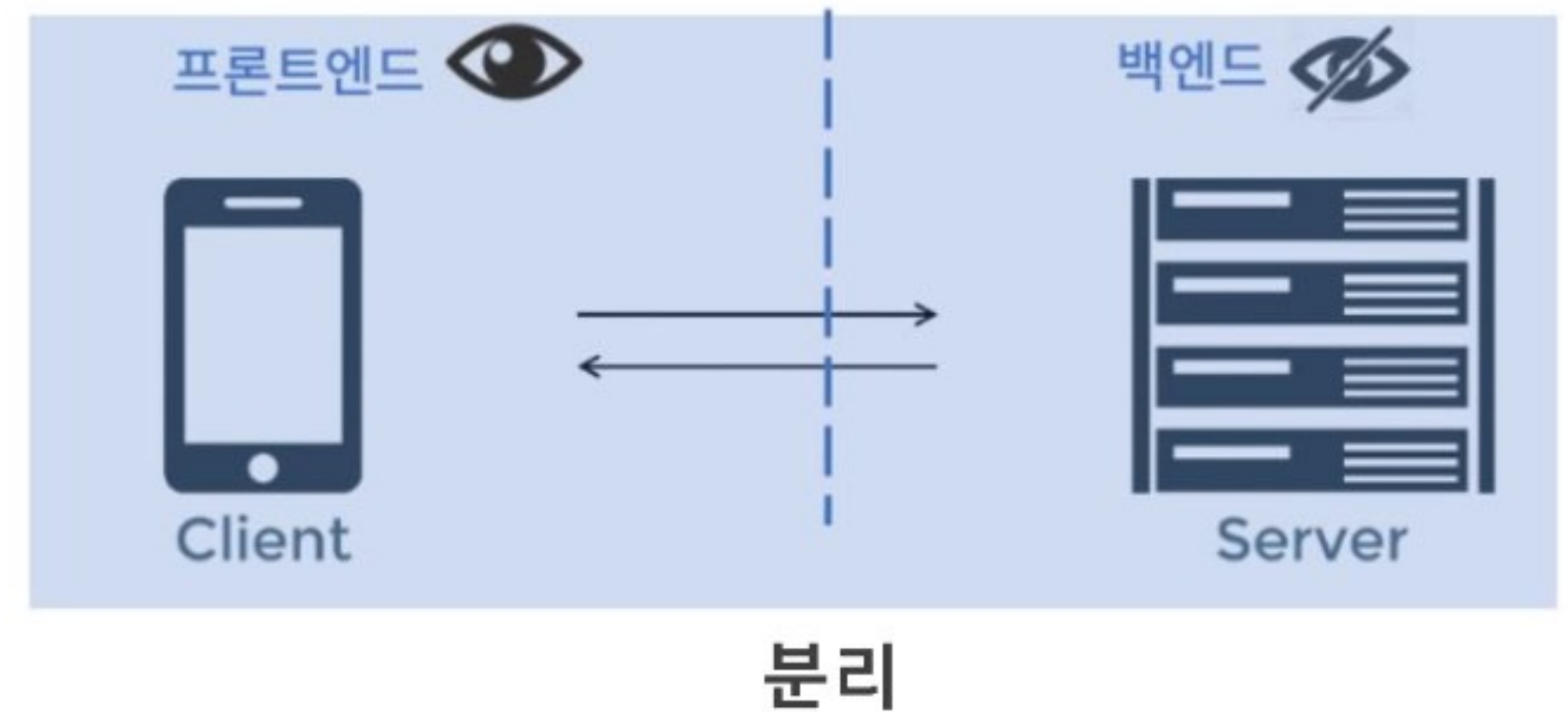
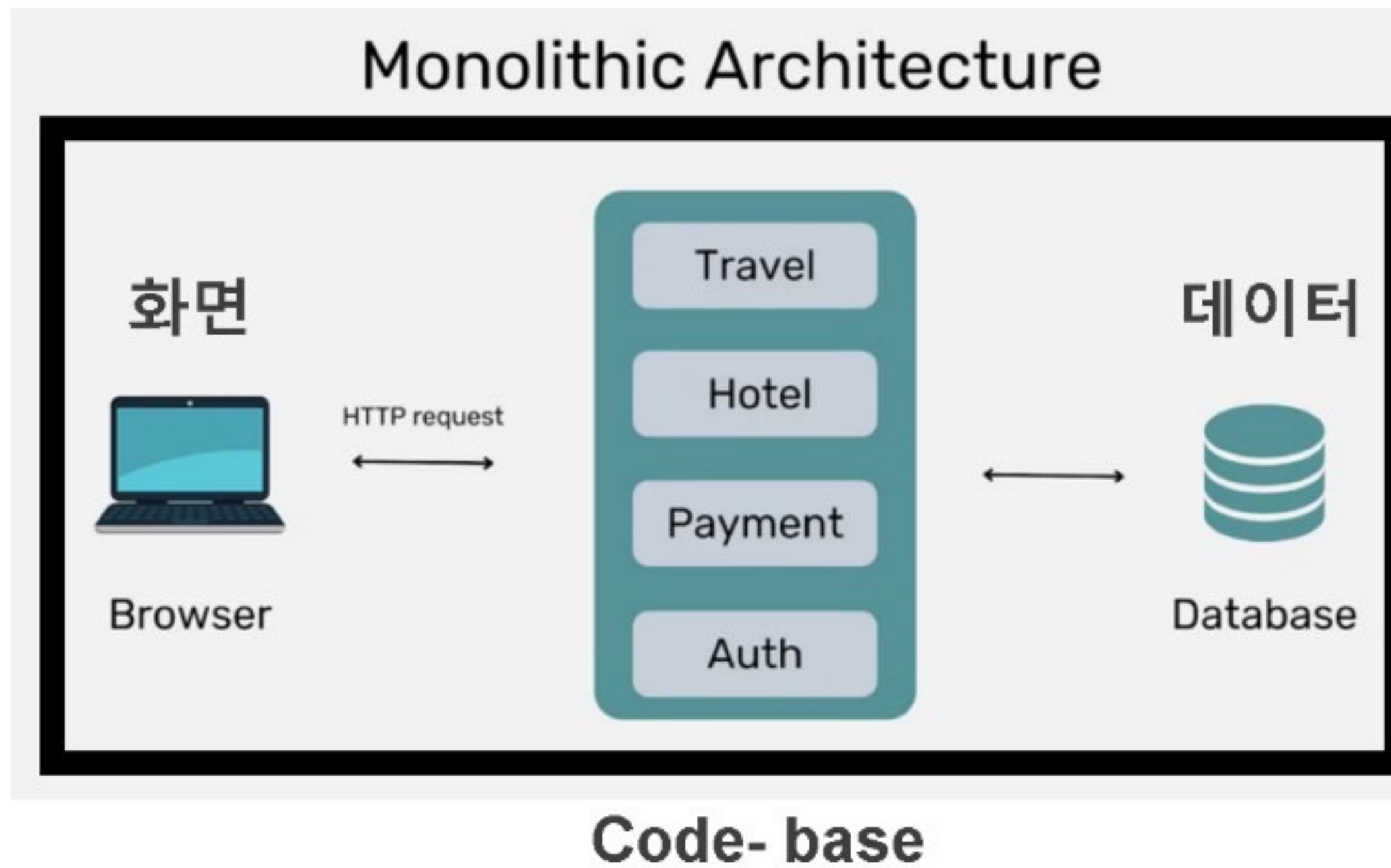


# 교육목표

- 백엔드 프론트엔드 이런 방식으로 통신해라!
- REST API 특징 알기

# REST API 등장 배경

## - Monolithic 방식



# Roy Fielding의 REST API



Microsoft Research

# REST란?

- URI로 식별된 자원의 상태 변화를 클라이언트와 서버가 주고받는다

## REpresentational State Transfer

“REST”

|        |             |                          |
|--------|-------------|--------------------------|
| GET    | /movies     | Get list of movies       |
| GET    | /movies/:id | Find a movie by its ID   |
| POST   | /movies     | Create a new movie       |
| PUT    | /movies     | Update an existing movie |
| DELETE | /movies     | Delete an existing movie |

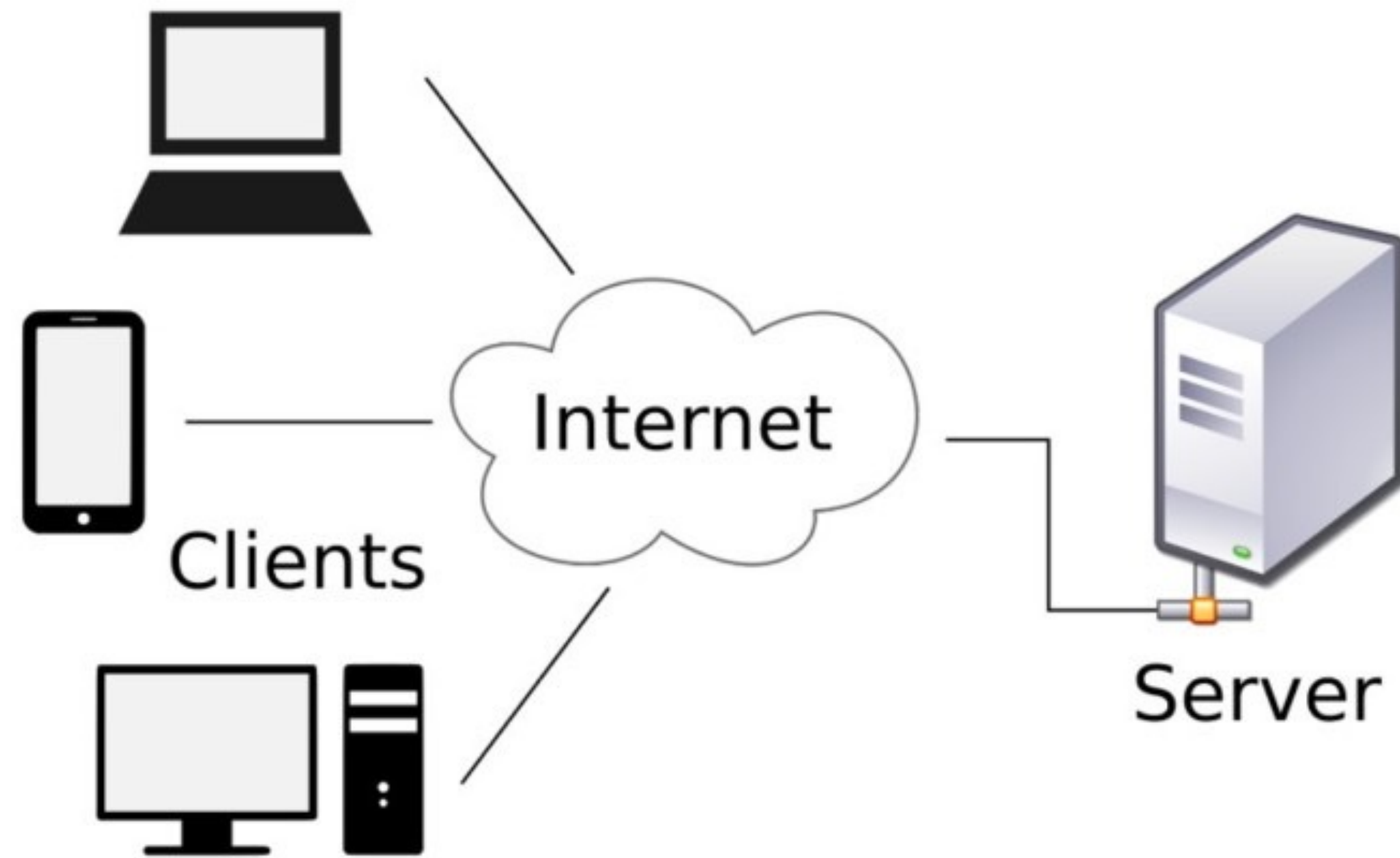
# REST 아키텍처란?

- 웹에서 데이터를 주고 받는 스타일

- client-server
- stateless
- cache
- **uniform interface (지켜지기 어려운 제약 조건)**
- layered system
- code-on-demand (optional)

# 1. 클라이언트 서버

리소스 : HTML, JSON, 이미지, 사람



## 2. Stateless 상태가 없는

EX) 로그인

NAVER

🔑 ID 로그인

[1] 일회용 번호

📄 QR코드

👤 아이디

🔒 비밀번호

☒ 로그인 상태 유지

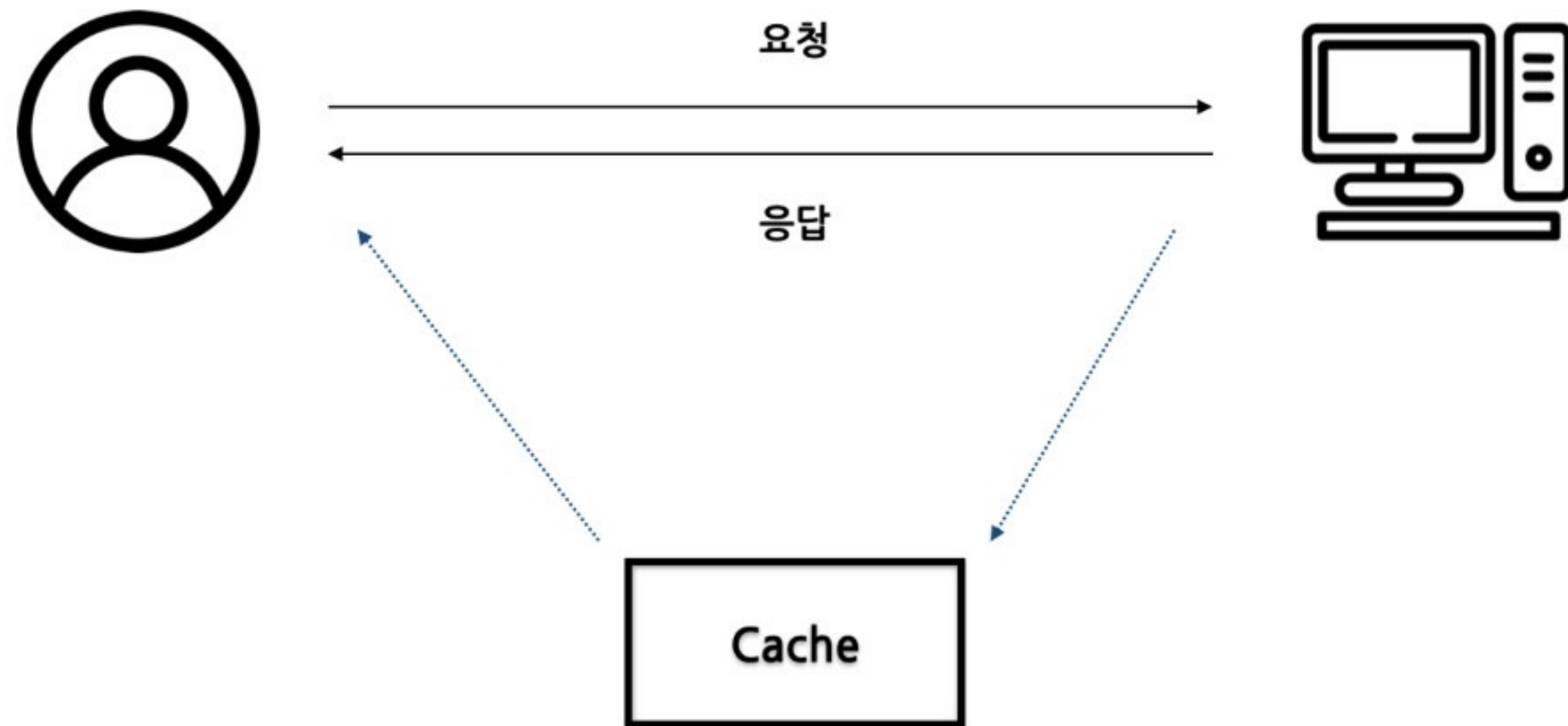
IP보안 ☐

로그인

[비밀번호 찾기](#) | [아이디 찾기](#) | [회원가입](#)

### 3. Cacheable 캐시되는 데이터

EX) HTTP Cache-Control





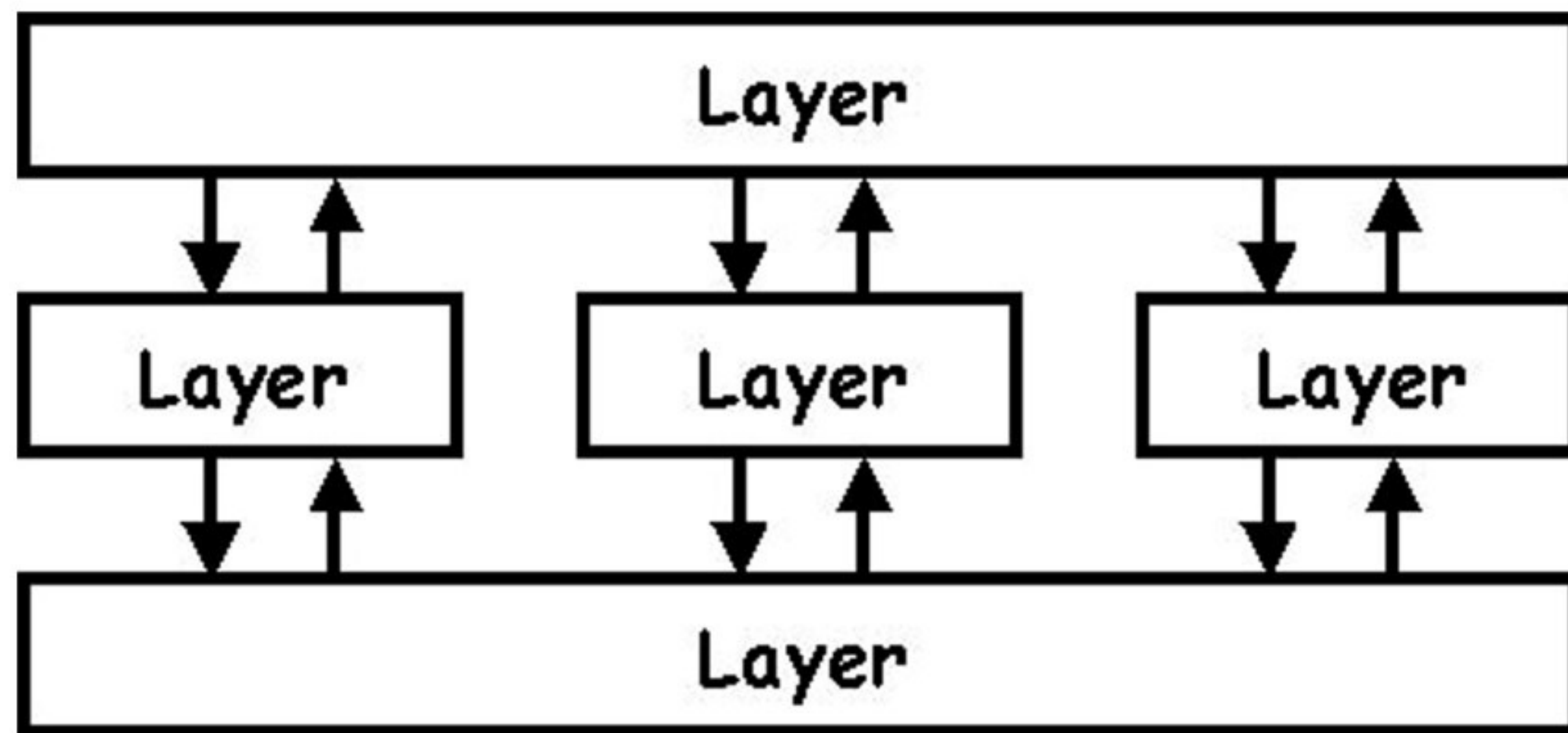
## 4. Uniform interface

1. 자원의 식별
2. 표현을 통한 자원의 조작
3. 자기 서술적 메시지 (Self-descriptive)
4. HATEOAS



## 5. Layered System

EX) 인증서버, 캐싱서버, 로드밸런서



## 6. Code-on-Demand

EX) javascript

# JavaScript



```
<html>
<head>
  <div>
    <div>
      <form method="post" action="#" id="formvalue" onkeyup="
drawChart()" />
    </form>
  </div>
</div>

<script type="text/javascript" src="https://www.google.com/jsapi"></
script>
<script type="text/javascript">

var bid = 43;
var ask = 21;

google.load("visualization", "1", {packages:["corechart"]});
google.setOnLoadCallback(drawChart);
function drawChart() {
  var data = google.visualization.arrayToDataTable([
    ['Price', 'Quantity'],
    ['Value #1', bid],
    ['Value #2', ask],
  ]);
```



# Uniform Interface

1. 자원의 식별
2. 표현을 통한 자원의 조작
3. 자기 서술적 메시지 (Self-descriptive)
4. HATEOAS



# 1. 자원의 식별



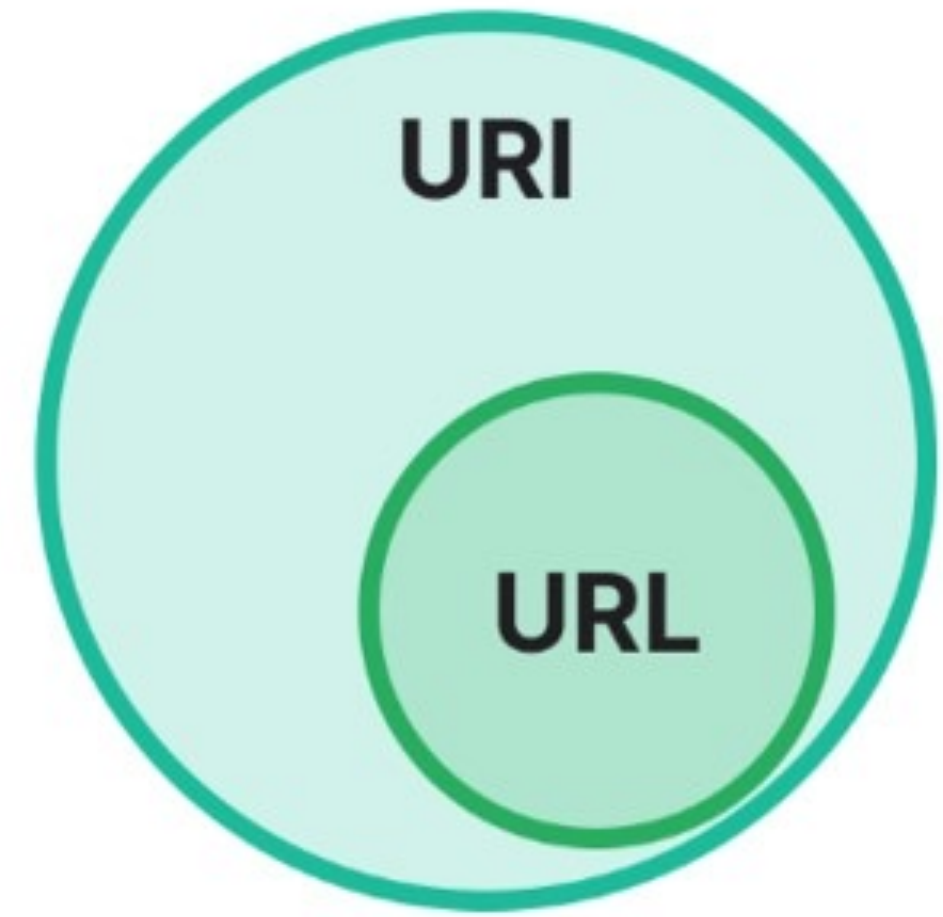
자원

>



객체

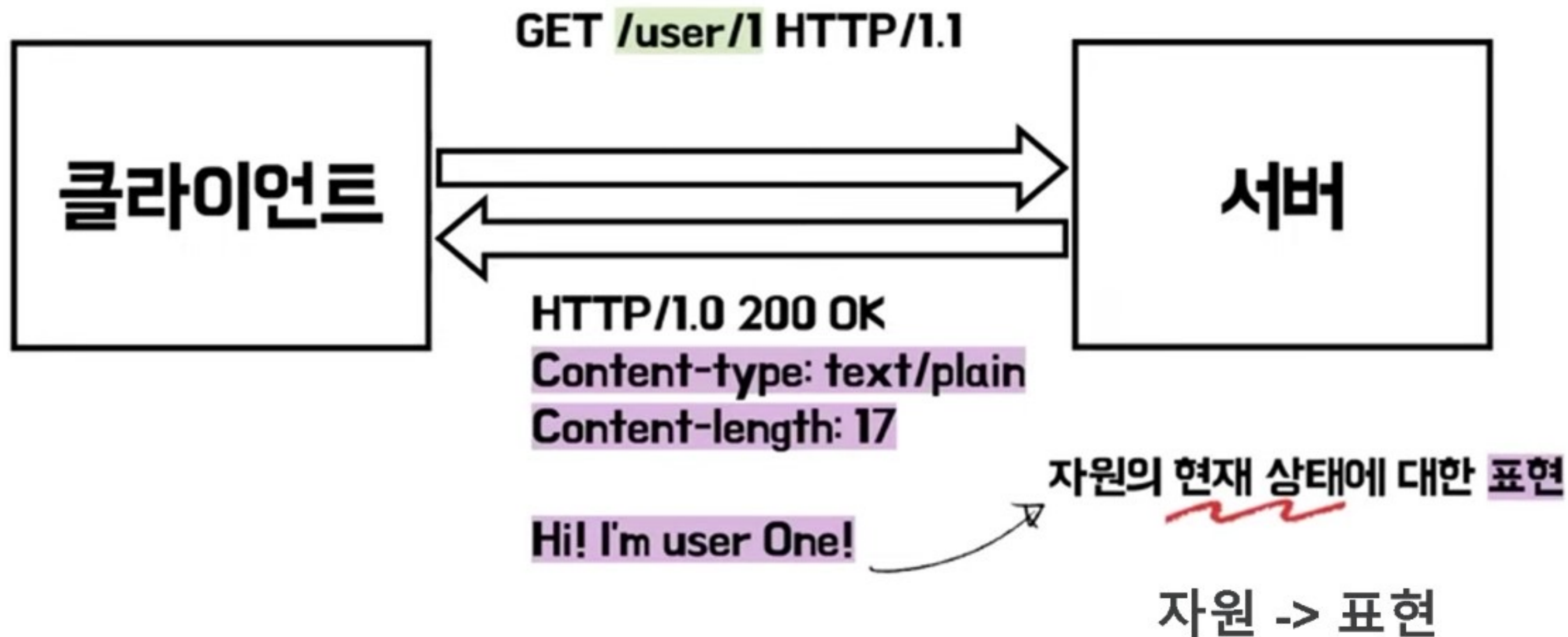
>



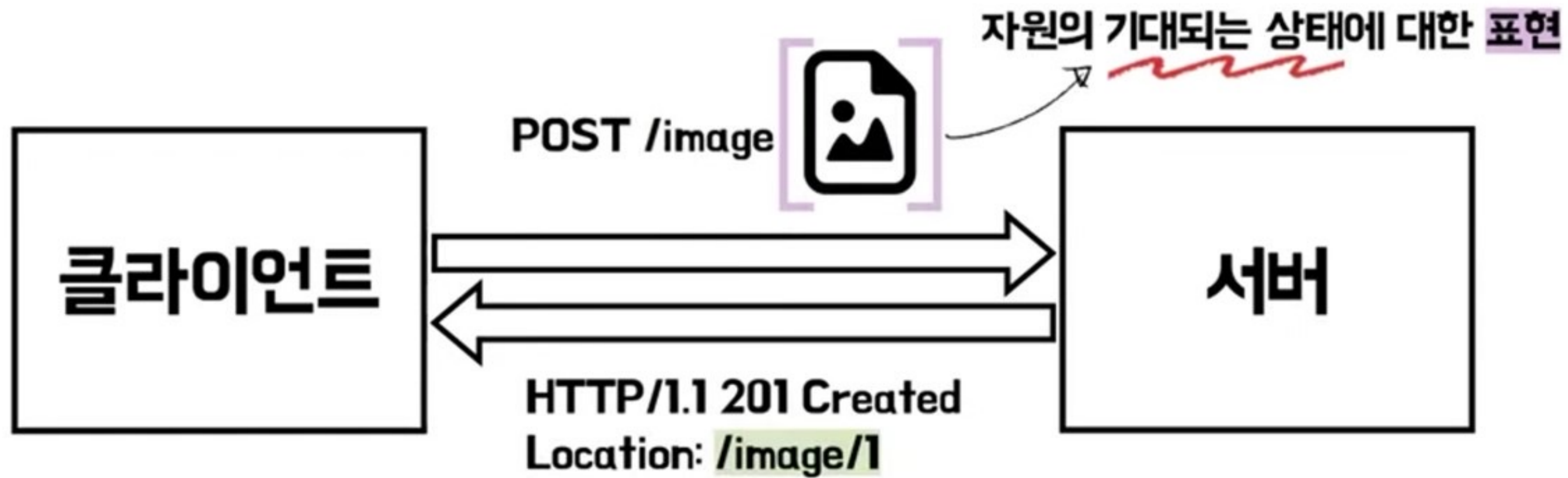
식별자, URI

## 2. 표현을 통한 자원의 조작 (1)

자원 : 다양한 방식으로 표현가능 ex) 문서, 파일, HTTP 엔티티



## 2. 표현을 통한 자원의 조작 (2)



표현 -> 자원

# Self-descriptive messages



Object

Component

Top

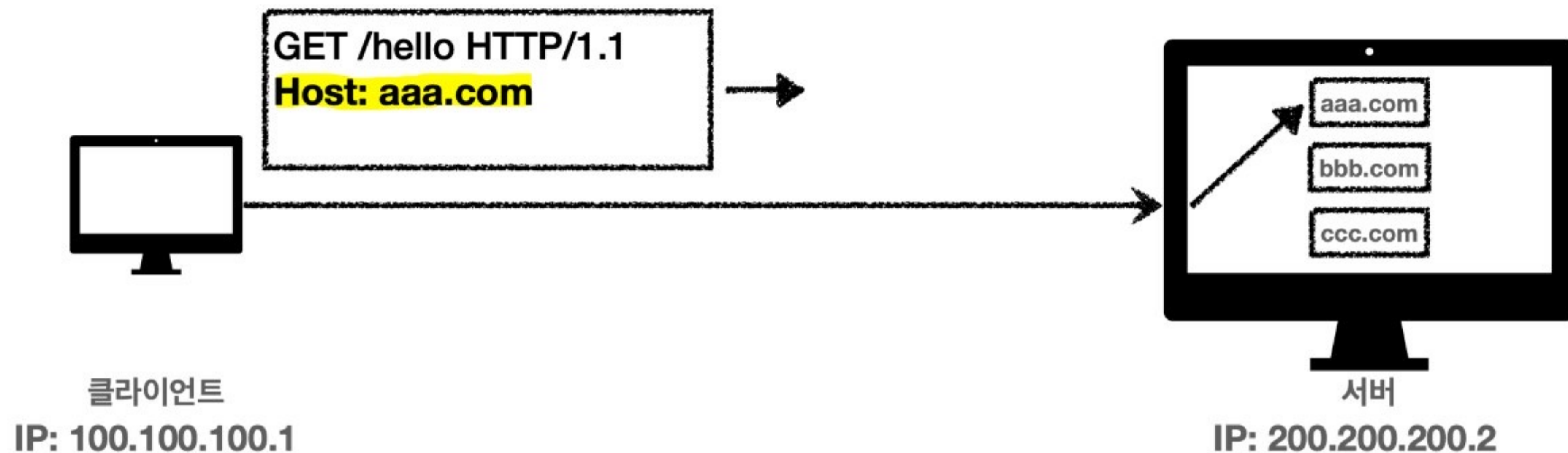
Right

Left

Bottom

# Self-descriptive messages - 도메인

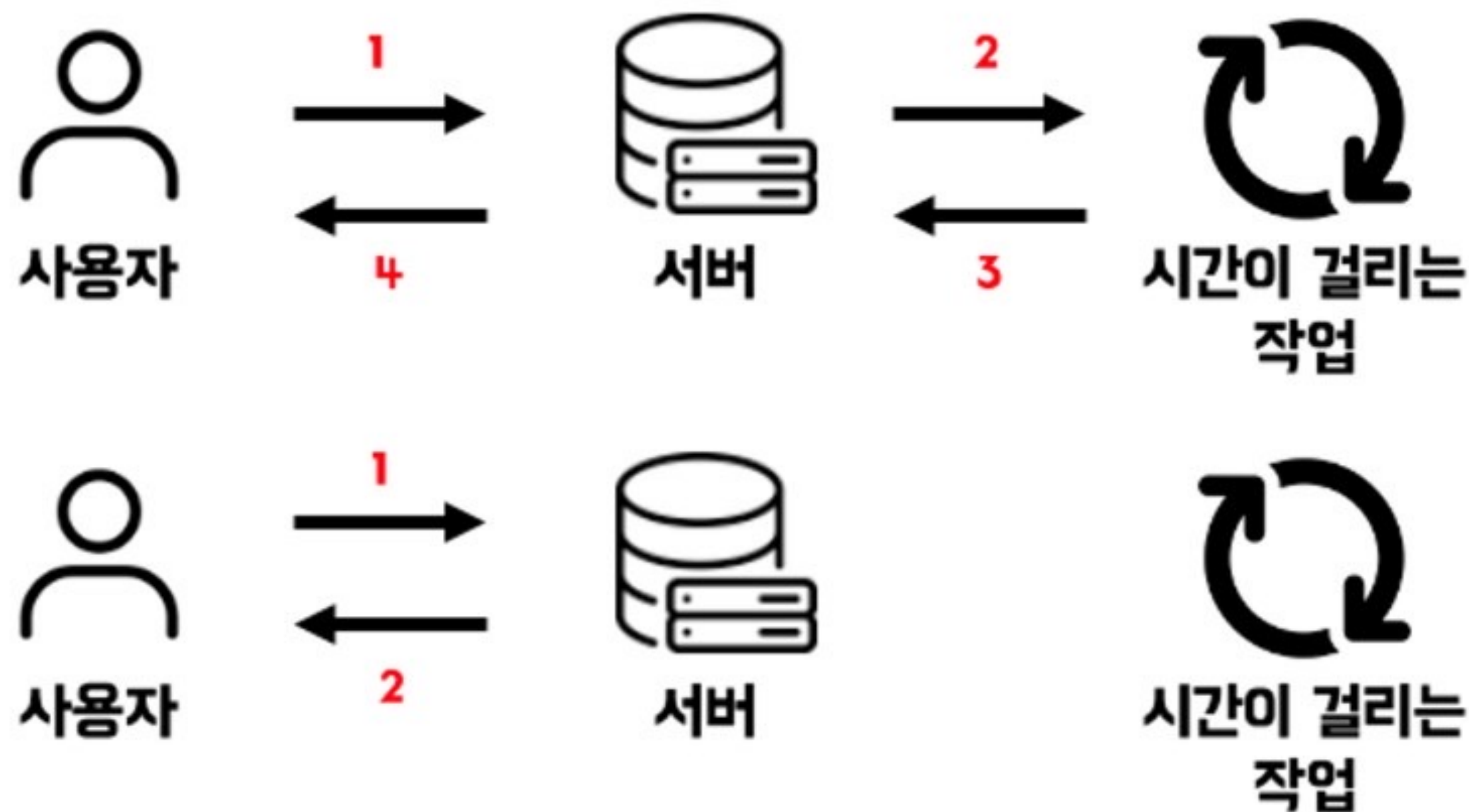
Host 헤더에 도메인명 기재 필요!  
HTTP/1.1 부터 HOST 헤더 필수화



# Self-descriptive messages - 캐쉬

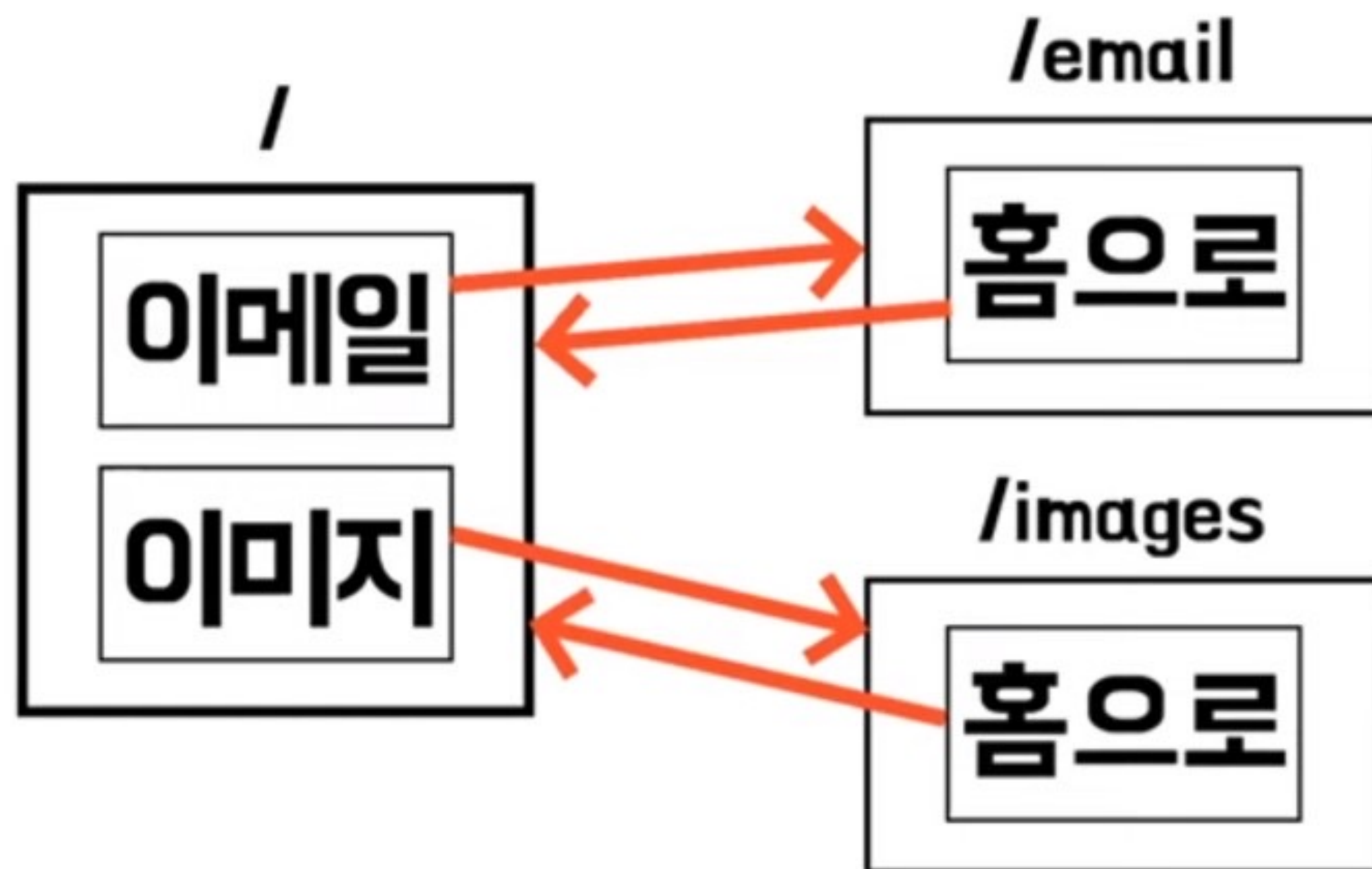
캐쉬 관련 헤더를 통한 캐쉬 전략 지정

HTTP/1.1 부터 Cache-Control, Age, Etag, Vary 태그 메시지



# HATEOAS

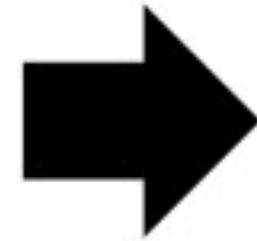
Hypermedia as the engine of application state



# HATEOAS

## <a> 태그 대신 URI 전달

```
{
  "DocumentType": 1,
  "No.": "S-ORD101001",
  "SellToCustNo": "10000",
  "PostingDate": "2023-04-02",
  "Lines": [
    {
      "LineNo": 10000,
      "Type": 2,
      "No": "1996-S",
      "Quantity": 12,
      "UnitPrice": 1397.3
    },
    {
      "LineNo": 20000,
      "Type": 2,
      "No": "1900-S",
      "Quantity": 4,
      "UnitPrice": 192.8
    }
  ]
}
```



← → ↻ ⓘ 127.0.0.1:8020/index/contents/main/test.html

네이버로 이동

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>test</title>

  </head>

  <body>
    <a href="http://www.naver.com">네이버로 이동</a>

  </body>
</html>
```

# 왜 REST API여야 하는가?

시스템을 완전히 통제할 수 있다고 생각한다면  
REST가 필요한지에 대해 시간 낭비하지 말아라!

- Roy Fielding -

## REST API 구현??

- ★ 1. REST API 구현하고 REST API라고 부르기 (도전)
- 2. REST API 구현을 포기하고 HTTP API라고 부르기
- 3. REST API가 아니지만 REST API라고 부르기 (현재 상태)



# Self-descriptive와 HATEOAS 만족시키기

## Self-descriptive

- Media type 문서
- Profile 링크 명시

## HATEOAS

- Json API와 같은 하이퍼링크 표현 명세 활용
- Link 등의 헤더로 링크를 표현



✦ CS QUIZ ✦



## Q.1

다음 중 REST API에 대한 설명으로 옳은 것을 모두 고르시오.

1. 클라이언트와 서버의 역할을 명확히 분리시킨다.
2. 웹에서 데이터를 주고받을 때 반드시 지켜야 할 표준이다.
3. URI를 통해 자원을 주고받을 수 있다.
4. HTTP 프로토콜을 사용하지 못하고 REST API를 사용하기 위한 별도의 인프라 구축이 필요하다.



## A.1

다음 중 REST API에 대한 설명으로 옳은 것을 모두 고르시오.

1. 클라이언트와 서버의 역할을 명확히 분리시킨다.
2. 웹에서 데이터를 주고받을 때 반드시 지켜야 할 표준이다.
3. URI를 통해 자원을 주고받을 수 있다.
4. HTTP 프로토콜을 사용하지 못하고 REST API를 사용하기 위한 별도의 인프라 구축이 필요하다.

1, 3

Q.2

REST의 약자를 적으시오.



## A.2

REST의 약자를 적으시오.

**REpresentational State Transfer**

## Q.3

다음 중 Uniform interface의 제약 조건이 아닌 것은?

- 1) HATEOAS
- 2) 표현을 통한 자원의 조작,
- 3) 자기 서술적 메시지,
- 4) 캐시되는 데이터



## Q.3

다음 중 Uniform interface의 제약 조건이 아닌 것은?

- 1) HATEOAS
- 2) 표현을 통한 자원의 조작,
- 3) 자기 서술적 메시지,
- 4) 캐시되는 데이터

4

## Q.4

### REST API 현재 상태는?

- 1) REST API를 구현하였고, REST API로 불린다
- 2) REST API가 아니지만, REST API로 부른다
- 3) REST API 구현을 포기하고, HTTP API라고 부른다
- 4) REST API를 구현하였고, HTTP API라고 부른다



## A.4

### REST API 현재 상태는?

- 1) REST API를 구현하였고, REST API로 불린다
- 2) REST API가 아니지만, REST API로 부른다
- 3) REST API 구현을 포기하고, HTTP API라고 부른다
- 4) REST API를 구현하였고, HTTP API라고 부른다

2

## Q.5

**Stateless(무상태성)에 대한 설명으로 옳지 않은 것은?.**

- 1) 서버는 클라이언트의 요청을 처리한 후 상태를 보관하지 않는다.
- 2) 이전 작업 내용을 다음 요청 시에 사용할 수 없다.
- 3) 서버에 저장된 정보를 이용해 이득을 취할 수 있다.
- 4) 세션의 정보는 전적으로 클라이언트가 가지고 있어야한다.



## A.5

**Stateless(무상태성)에 대한 설명으로 옳지 않은 것은?.**

- 1) 서버는 클라이언트의 요청을 처리한 후 상태를 보관하지 않는다.
- 2) 이전 작업 내용을 다음 요청 시에 사용할 수 없다.
- 3) 서버에 저장된 정보를 이용해 이득을 취할 수 있다.
- 4) 세션의 정보는 전적으로 클라이언트가 가지고 있어야한다.

3

## Q.6

전체적인 시스템 아키텍처를 간단하고 잘 파악할 수 있도록 REST한 규약을 사용자들이 지킴으로서 client 개발자와 server 개발자간의 결합도를 낮추는 제약은?

- 1) 캐시되는(Cacheable) 데이터
- 2) 일관적인 인터페이스(Uniform interface)
- 3) 레이어드 시스템 (Layered System)
- 4) Code-on-demand

## A.6

전체적인 시스템 아키텍처를 간단하고 잘 파악할 수 있도록 REST한 규약을 사용자들이 지킴으로서 client 개발자와 server 개발자간의 결합도를 낮추는 제약은?

- 1) 캐시되는(Cacheable) 데이터
- 2) 일관적인 인터페이스(Uniform interface)
- 3) 레이어드 시스템 (Layered System)
- 4) Code-on-demand

2

## Q.7

자기 서술적 메시지(Self-descriptive messages)에서 GET 요청 메시지의 경우 0000 00에 도메인명을 기재해야 한다.



A.7

자기 서술적 메시지(Self-descriptive messages)에서 GET 요청 메시지의 경우 0000 00에 도메인명을 기재해야 한다.

Host 헤더

## Q.8

**REST 아키텍처 스타일의 제약조건이 아닌 것은?**

- 1) Layered System
- 2) Cache
- 3) 인터페이스의 무작위성
- 4) Stateless



## A.8

REST 아키텍처 스타일의 제약조건이 아닌 것은?

1) Layered System

2) Cache

3) 인터페이스의 무작위성

4) Stateless

## Q.9

REST 아키텍처에서 클라이언트와 서버 간의 동적인 상호작용을 가능하게 하는 요소는 무엇인가요?



A.9

REST 아키텍처에서 클라이언트와 서버 간의 동적인 상호작용을 가능하게 하는 요소는 무엇인가요?

**HATEOAS**



## Q.10

코테이토에서 조감자는 프론트엔드로, 박감자는 백엔드로 웹 프로젝트를 진행하고 있다. API를 이용해 클라이언트와 서버간 리소스를 공유하고 있고, 모든 API 요청에는 사용자가 로그인했는지 알려주는 정보를 포함해서 보낸다. 다음 웹 페이지가 준수하고 있는 REST 아키텍처를 모두 고르시오.

1. client server
  2. stateless
  3. cacheable
  4. layered system
- 

## A.10

코테이토에서 조감자는 프론트엔드로, 박감자는 백엔드로 웹 프로젝트를 진행하고 있다. API를 이용해 클라이언트와 서버간 리소스를 공유하고 있고, 모든 API 요청에는 사용자가 로그인했는지 알려주는 정보를 포함해서 보낸다. 다음 웹 페이지가 준수하고 있는 REST 아키텍처를 모두 고르시오.

- 1. client server
- 2. stateless
- 3. cacheable
- 4. layered system

1, 2



◆ CS EDUCATION ◆



Thank you

